

ECE 473/573
Cloud Computing and Cloud Native Systems
Lecture 01 Introduction

Professor Jia Wang
School of Engineering, Applied Science, and Health
Illinois Institute of Technology

August 17, 2026

Outline

Administrative Issues

AI-Assisted Software Development

Reading Assignment

- ▶ This lecture: Course Syllabus
- ▶ Next lecture: 1, 6

Outline

Administrative Issues

AI-Assisted Software Development

- ▶ Professor Jia Wang
- ▶ E-Mail: jwang34@illinoistech.edu
- ▶ Office hours: Please email me directly.

Lectures

- ▶ Mon./Wed. 1:50 PM – 3:05 PM
- ▶ Mon. Hermann Hall 005
- ▶ Wed. IIT Tower 6C7-1
- ▶ Course website:
<https://wngjia.github.io/ece573-web/>

Textbooks

- ▶ Required Textbook
 - ▶ “Cloud Native Go: Building Reliable Services in Unreliable Environments”, Matthew Titmus, O'Reilly Media, 2021.
- ▶ Plus additional research papers

Useful Websites

- ▶ <https://www.youtube.com>
 - ▶ Find tutorials.
- ▶ <https://stackoverflow.com>
 - ▶ Learn how to communicate with professionals.

Prerequisite

- ▶ Computer programming
 - ▶ Branch and loop, function
 - ▶ Class
 - ▶ Searching and sorting
 - ▶ Debugging and troubleshooting
- ▶ Computer organization
 - ▶ Memory and pointer
 - ▶ Interrupts
 - ▶ I/O subsystems for storage and communication

Software system design for cloud

- ▶ Cloud computing architecture
 - ▶ Virtualization, containerization, and resource management
 - ▶ Distributed storage, communication, and computing systems
- ▶ Cloud native system design and implementation
 - ▶ Scalability, resilience, manageability, observability
 - ▶ Go language introduction and common cloud native patterns

Course Objectives (ABET)

After completing this course, the student should be able to do the following:

1. Identify scenarios where cloud computing applies.
2. Understand virtualization and containerization techniques.
3. Explain common storage, communication, and computing patterns in cloud.
4. Describe cloud native system design techniques for security, scalability, resilience, manageability, and observability.
5. Utilize open-source software to build cloud native systems.

Homeworks/Projects

- ▶ 5 Homeworks
 - ▶ 2 points each for a total of 10 points
- ▶ 7 Projects
 - ▶ Project 1: 10 points
 - ▶ Project 3 and 7: 15 points
 - ▶ Project 2, 4, 5, and 6: 20 points
 - ▶ For a total of 120 points
- ▶ Submit online in Canvas only.

ECE 473 Grading

- ▶ A: 90
- ▶ B: 80
- ▶ C: 60
- ▶ D: 55

ECE 573 Grading

- ▶ A: 115
- ▶ B: 100
- ▶ C: 80

Expectations and Late Policy

- ▶ This is a project based course and you are expected to practice your problem solving and troubleshooting skills.
- ▶ Learn how to manage deadlines. **Do NOT expect to finish a project during the weekend right before its deadline.**
- ▶ Late homeworks and projects will NOT be graded, unless
 - ▶ A request to extend the deadline is received by email **48 hours BEFORE the deadline.**
 - ▶ Within 48 hours of the deadline or after, the request should be accompanied by **extraordinary reasons with documented proof like doctor's notes, or it will be rejected.**
- ▶ **Extraordinary reasons do NOT include**
 - ▶ Loss of code or progress or report due to software, hardware, and networking failures.
 - ▶ AI assistants generate code that doesn't work.
- ▶ Resubmission/regrading of projects are not allowed.

Project Setup

- ▶ A recent Windows computer with 4 CPU cores, 16GB memory, and 512GB SSD.
 - ▶ Or access to a x64 Ubuntu server with 4 CPU cores, 16GB memory, and 100GB storage.
 - ▶ We are not able to support ARM-based computers, like Apple MacBooks and Raspberry Pi's, since many docker images don't support ARM.
 - ▶ We are not able to support computers that are more than 5 years old since virtualization would require substantial amount of computational and I/O power.
- ▶ Internet access is required to access common code and package repositories like GitHub and Docker Hub.
- ▶ (Recommended) Subscription to coding agent plans like Claude Code or Codex.

How to survive succeed in this course?

- ▶ Read: all instructions are in writing.
 - ▶ Tutorials, source code, documents, and **don't overlook command outputs**.
- ▶ Communicate: we are very happy to solve any issue you may meet but you need to let us know what's wrong.
 - ▶ <https://stackoverflow.com/help/how-to-ask>
- ▶ Surprisingly, these are also the skills you need the most when working with AI tools.
- ▶ Feel free to explore new computer hardware and software but make sure they do not interfere with your schedule to meet deadlines.

Ethics (Very Seriously)

- ▶ Read “IIT Code of Academic Honesty” and “IEEE Code of Conduct” (posted on the course website).
 - ▶ Projects/homeworks should be done individually.
 - ▶ Discussions on homeworks/projects are encouraged.
 - ▶ Interactions with AI tools (prompts and answers) should not be shared since they are considered as your own work.
 - ▶ Source code from the lectures and instructions in this course can be used directly.
 - ▶ Source code from other online sources not directly related to this course may be used with proper references.
- ▶ All other writings and code should be **BY YOURSELF**.
 - ▶ NEVER SHARE YOUR WRITINGS/CODE WITH OTHERS!
 - ▶ NEVER USE WRITINGS/CODE FROM OTHERS!
 - ▶ NEVER POST YOUR PROJECT CODE OR ASK FOR HELP DIRECTLY ONLINE!
- ▶ Please review our **Academic Honesty Guidelines**.

<https://www.iit.edu/academic-affairs/academic-honesty-guidelines>

Outline

Administrative Issues

AI-Assisted Software Development

The Revolution is Here

- ▶ Late 2022: Chatbots powered by large language models (LLMs) become publicly available.
- ▶ Early 2025: Vibe Coding
 - ▶ LLMs generate code from simple descriptions, enabling people with no programming experience to create useful programs.
 - ▶ It is difficult to improve these programs further with LLMs directly because of their limitations.
- ▶ Late 2025: Coding Agent
 - ▶ With a proper harness system, LLMs are able to work on complex projects and interact with software developers.
 - ▶ Recently, Anthropic reported that 80% code are authored by Claude, and engineers are shipping 8X as much code.
 - ▶ Many software developers claim to have 90%, if not all, code written by coding agents since early 2026.
- ▶ Please complete the AI-Assisted Programming and Software Development Survey on Canvas in Quizzes.

A Few Examples

- ▶ “Write a function in Java to implement quicksort.”
 - ▶ Most LLMs should be able to complete this simple task directly because quicksort is a well-known algorithm.
- ▶ “Write a Flappy Bird game in Python.”
 - ▶ LLMs learn the requirements of this game through training and should be able to complete this more complicated task.
- ▶ “Write a SQL query to locate user account in our database.”
 - ▶ While LLMs generally know how to write a SQL query, they need to know your database tables and columns first.
 - ▶ Can LLMs discover that information automatically?
- ▶ “Write a shopping website.”
 - ▶ LLMs may generate a working shopping website but further interactions and improvements are necessary to fit your need.
 - ▶ How to communicate your need efficiently to LLMs?

Language Models

- ▶ Tokenization: how to represent texts in natural language?
 - ▶ Represent texts as a sequence of tokens.
 - ▶ Subword tokenization: handle common character patterns with meaning as well as rare/new/misspelled words.
- ▶ Language model: $Pr(t_{n+1}) = D(t_1, t_2, \dots, t_n)$
 - ▶ Learn D to predict the probability of the next token t_{n+1} from a sequence of input tokens $t_1, t_2, \dots, t_n$.
 - ▶ Sample $Pr(t_{n+1})$ to obtain the actual next token t_{n+1} .
- ▶ Autoregression: how to generate sequence of tokens?
 - ▶ Feed the generated t_{n+1} back into the model to generate t_{n+2} .
 - ▶ $Pr(t_{n+2}) = D(t_1, t_2, \dots, t_{n+1})$
- ▶ Context window: how to handle arbitrary sequence lengths?
 - ▶ It is difficult to train models for arbitrary sequence lengths.
 - ▶ Instead, the model only inputs a sequence of at most N tokens.
 - ▶ Most frontier models have $N = 1\text{million}$ today.

LLM Code Generation

- ▶ LLMs are trained to follow your instruction to generate code, just like how they answer your question in natural language.
- ▶ But much more is needed than your instruction.
 - ▶ Additional information like database table names.
 - ▶ Existing lines of code if they need to be modified.
 - ▶ Make sure to fit all inputs and outputs into the context window so the model won't miss any.
 - ▶ Write generated code to the source files.
- ▶ What if there is a bug in the generated code or if it simply won't compile?
 - ▶ The statistical nature of the generation process makes it impossible to guarantee 100% correctness.
- ▶ Software developers can do all the above manually.
 - ▶ But not efficient.
 - ▶ Can LLMs take care of them automatically?

- ▶ Structured output and tool calling
 - ▶ LLMs are trained to generate texts in specific formats like json.
 - ▶ If LLMs encode their intents (e.g. read a file) in such formats, a software program can parse them and complete the tasks.
- ▶ Agents: software programs that interact with LLMs.
 - ▶ Prepare and send instructions to LLMs based on human input.
 - ▶ If LLMs generate output to call tools, e.g. to read a file, agents call them and use tool outputs to form follow-up instructions toward additional iterations.
 - ▶ Until LLMs don't require further tool calls – at this time, agents can accept further human instructions.

Summary

- ▶ An LLM generates text and code based on the model and the information in its context window.
- ▶ Coding agents extend LLMs with the ability to use tools, e.g. to read/write files and to run commands.
- ▶ When building cloud and cloud-native systems, what are the key aspects that require human judgment?